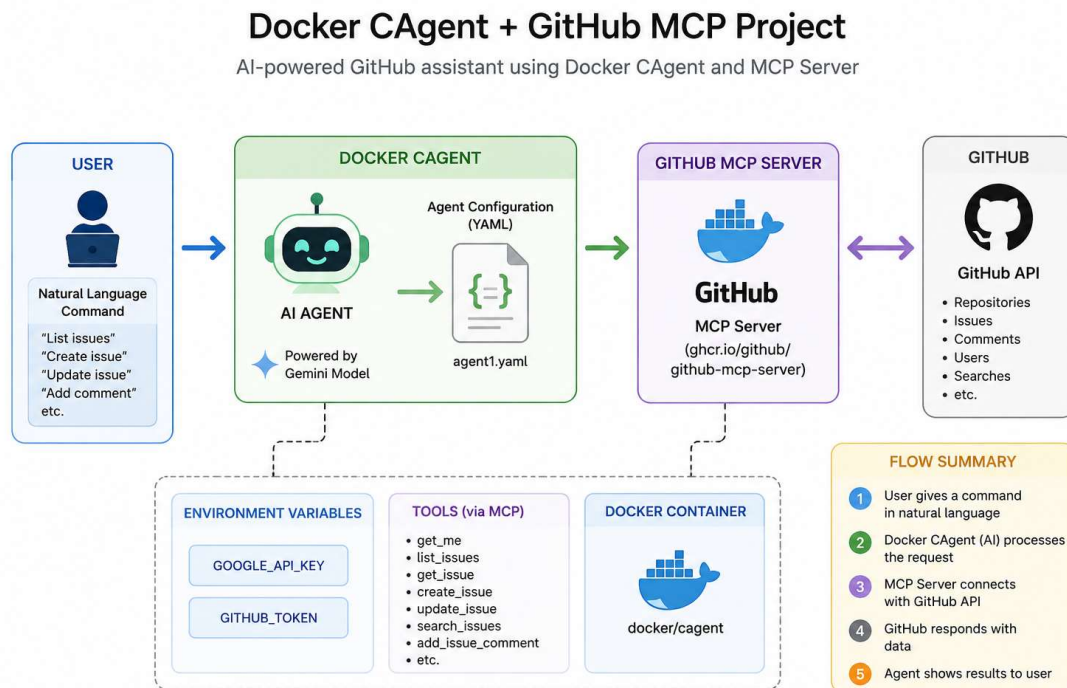


Docker CAgent with GitHub MCP Integration

Docker CAgent is an AI-powered command-line assistant that helps users automate development and DevOps tasks using natural language. Instead of manually writing complex commands, users can interact with the agent like chatting with an assistant. In this project, Docker CAgent was integrated with the GitHub MCP (Model Context Protocol) Server to allow the AI agent to communicate directly with GitHub repositories. This integration enables tasks such as listing issues, creating issues, updating repositories, and managing GitHub workflows through simple text commands. The project demonstrates how AI agents, Docker containers, and MCP integrations can simplify real-world DevOps and GitHub management operations in an efficient and user-friendly way.



github.com/shivaji67/cagent-AI-Project

Objective

The objective of this task was to configure Docker CAgent and integrate it with the GitHub MCP Server to create an AI-powered

linkedin.com/in/shivaji-bedre/

GitHub assistant capable of handling GitHub operations using natural language commands.

Pull Docker Images

```
PS C:\Users\HP\genai> docker pull docker/cagent:latest
latest: Pulling from docker/cagent
Digest: sha256:2c69f321a57832cd3c4b38a79194508e6fd48fd20994536f9307cdfed247689
Status: Image is up to date for docker/cagent:latest
docker.io/docker/cagent:latest
PS C:\Users\HP\genai> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
docker/cagent:latest	2c69f321a578	242MB	61.2MB	
ghcr.io/github/github-mcp-server:latest	078df53ba8b2	60.6MB	15.2MB	
hello-world:latest	05813aedc15f	25.9kB	9.52kB	U
mysql:latest	57c892ddeb17	1.29GB	293MB	
nginx:latest	df281a66ef12	240MB	65.8MB	U
postgres:latest	5773fe724c49	649MB	168MB	
shiva67/apache-ubuntu-demo:latest	e6e9c22778e2	367MB	105MB	
ubuntu:latest	7a398104c5a2	119MB	31.7MB	U
wordpress:latest	afa48d92726d	1.06GB	272MB	

```
PS C:\Users\HP\genai> |
```

The required Docker images were downloaded successfully using Docker commands.

Create Agent Configuration Files

Two YAML configuration files were created:

- agent.yaml → Linux/Docker assistant
- agent1.yaml → GitHub MCP assistant

```
PS C:\Users\HP\genai> cat .\agent.yaml
agents:
  root:
    description: Linux and Docker assistant
    instruction: |
      You are an expert in Linux, Docker, Kubernetes and DevOps.
    model: auto
```

These files define:

- AI model configuration
- MCP integration
- GitHub tools
- Agent instructions

Configure GitHub MCP Integration

```
PS C:\Users\HP\genai> cat .\agent1.yaml
version: "1"

agents:
  root:
    model: auto

    description: "GitHub Assistant"

    instruction: |
      You are a GitHub assistant.
      ALWAYS use GitHub MCP tools for GitHub operations.

    toolsets:
      - type: mcp

        command: docker

        args:
          - run
          - -i
          - --rm
          - -e
          - GITHUB_PERSONAL_ACCESS_TOKEN=${GITHUB_TOKEN}
          - ghcr.io/github/github-mcp-server

        tools:
          - get_me
          - list_issues
          - get_issue
          - create_issue
          - update_issue
          - search_issues
          - add_issue_comment
```

The agent1.yaml file was updated with:

- GitHub MCP server configuration
- GitHub token forwarding
- MCP tools configuration

This enabled the AI agent to communicate with GitHub services.

Run Docker CAgent

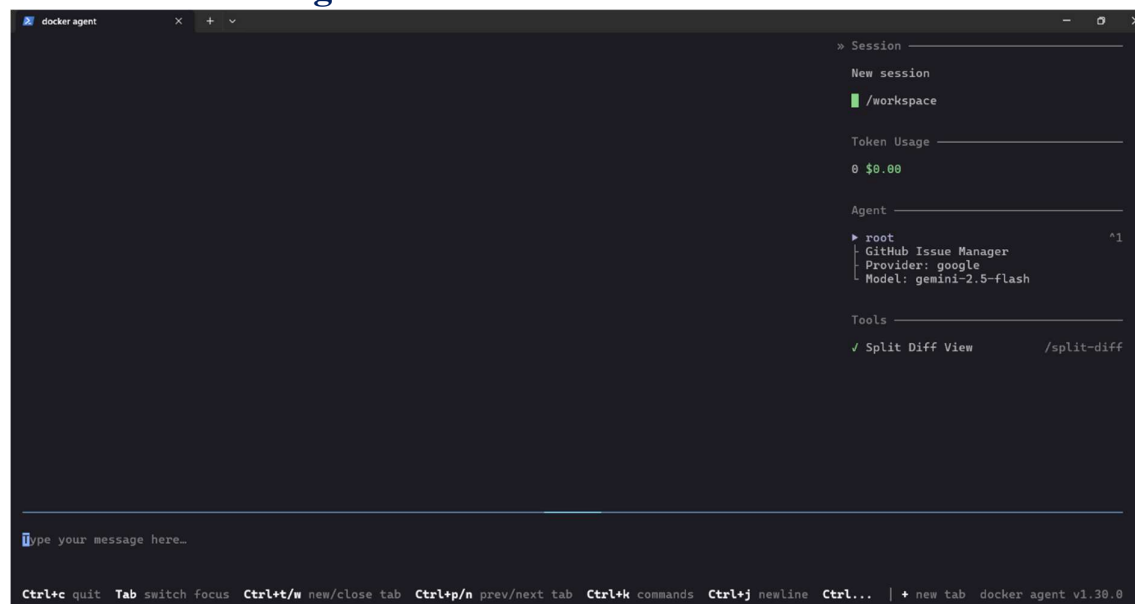
```
PS C:\Users\HP> cd .\genai\  
PS C:\Users\HP\genai> docker run -it --rm `   
>> -v ${PWD}:/workspace `   
>> -w /workspace `   
>> -e GOOGLE_API_KEY="AIzaSyBwwTNJwbx3b9H8oWvJu6qdJZHr3tUf0g|" `   
>> -e GITHUB_TOKEN="ghp_601NhtdimeMw9WL7Ph9JvI5d2pws4L4fFSz" `   
>> docker/cagent run agent1.yaml
```

The Docker CAgent container was executed using PowerShell with:

- Google Gemini API key
- GitHub token
- Workspace mounting
- YAML configuration

This started the AI-powered GitHub assistant.

Launch Docker Agent Interface

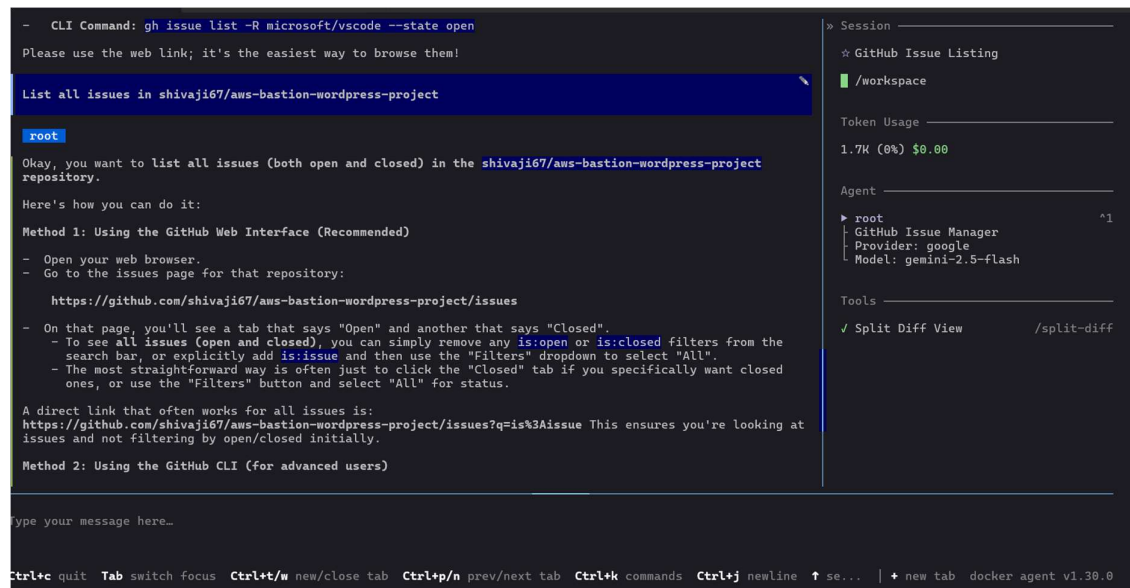


The Docker Agent interface started successfully.

The interface displayed:

- Active session
- Gemini model connection
- Available tools
- Workspace environment

Test GitHub Issue Commands



```
- CLI Command: gh issue list -R microsoft/vscode --state open
Please use the web link; it's the easiest way to browse them!

List all issues in shivaji67/aws-bastion-wordpress-project

root
Okay, you want to list all issues (both open and closed) in the shivaji67/aws-bastion-wordpress-project repository.
Here's how you can do it:
Method 1: Using the GitHub Web Interface (Recommended)
- Open your web browser.
- Go to the issues page for that repository:
  https://github.com/shivaji67/aws-bastion-wordpress-project/issues
- On that page, you'll see a tab that says "Open" and another that says "Closed".
  - To see all issues (open and closed), you can simply remove any is:open or is:closed filters from the search bar, or explicitly add is:issue and then use the "Filters" dropdown to select "All".
  - The most straightforward way is often just to click the "Closed" tab if you specifically want closed ones, or use the "Filters" button and select "All" for status.
A direct link that often works for all issues is:
https://github.com/shivaji67/aws-bastion-wordpress-project/issues?q=is%3Aissue This ensures you're looking at issues and not filtering by open/closed initially.
Method 2: Using the GitHub CLI (for advanced users)

Type your message here...

Ctrl+c quit  Tab switch focus  Ctrl+t/w new/close tab  Ctrl+p/n prev/next tab  Ctrl+k commands  Ctrl+j newline  ↑ se... | + new tab  docker agent v1.38.0
```

Session

- GitHub Issue Listing
- /workspace
- Token Usage: 1.7K (0%) \$0.00
- Agent
 - root
 - GitHub Issue Manager
 - Provider: google
 - Model: gemini-2.5-flash
- Tools
 - Split Diff View /split-diff

Natural language GitHub issue commands were tested inside Docker CAgent.

The agent responded to repository-related issue queries and GitHub operations.

This confirmed:

- Successful MCP integration
- AI agent functionality
- GitHub communication workflow

Challenges Faced

During the implementation, several issues were encountered:

- YAML parsing errors
- PowerShell encoding problems
- Incorrect environment variable configuration
- Smart quote issues in PowerShell commands
- GitHub MCP authentication setup

These issues were resolved through proper YAML formatting, ASCII encoding, and correct token forwarding configuration.

Conclusion

This task successfully demonstrated how to integrate Docker CAgent with the GitHub MCP Server to build an AI-powered GitHub assistant.

The project helped in understanding:

- Docker-based AI agents
- MCP architecture
- GitHub automation workflows
- YAML configuration
- AI-assisted DevOps integration

The final setup provided a functional environment where GitHub operations could be managed using simple natural language commands.